Relatório completo do projeto SGC

Guia técnico, análise do código-fonte e material de apoio à apresentação

DISCIPLINA	TURMA
GAC124 — Introdução aos Algoritmos	10R1
DISCENTES	**VERSÃO DO RELATÓRIO**
Jhônata de Oliveira Marques Henrique César Carvalho Aguiar	28 de outubro de 2026 Baseado no main.cpp revisado

Resumo executivo

O Sistema de Gerenciamento de Clientes é um programa de terminal em C++ que organiza cadastros em um vetor dinâmico, oferece operações de inclusão, consulta, alteração e remoção, e mantém os dados em arquivo binário com importação e exportação CSV. O relatório explica a organização do código, os tipos de dados, a entrada, o gerenciamento de memória, os algoritmos, a persistência, a interface e inclui uma matriz que liga cada requisito técnico aos trechos correspondentes do main.cpp.

REVISÃO TÉCNICA

As buscas binárias por ID e por nome são recursivas; ordenação e busca por nome compartilham a comparação; a matriz técnica também registra diferenças entre o comportamento atual e alguns detalhes do enunciado, incluindo capacidade inicial/crescimento e listagem por intervalo escolhido.

Escopo das verificações

- Compilação com g++ -std=c++11 -Wall -Wextra -pedantic: concluída sem avisos.
- Busca recursiva testada com IDs encontrados e ausentes, vetor vazio, nomes com capitalização diferente e nome ausente.
- A revisão exercitou diretamente as funções de busca. Não foi executada uma sessão completa de navegação por todos os menus.
- O plano de ensino de GAC124 inclui chamadas recursivas de funções e registra recursão junto ao conteúdo de busca binária.

Como usar este documento: os capítulos seguintes explicam o código para estudo e apresentação. O objetivo é compreender a função de cada parte e justificar as escolhas, sem decorar linhas.

Relatório formal do trabalho

Introdução

O trabalho consiste em implementar um sistema de cadastro em C++ com busca, alteração, ordenação e persistência em arquivos. O projeto escolhido administra clientes, permite consultar e modificar registros pelo terminal e mantém uma base inicial com 100 clientes.

Descrição de alto nível das estruturas e da lógica

O programa representa cada cliente por um registro que agrupa identificador, nome, endereço, ano de nascimento, documento, categorias e limite de crédito. Uma segunda estrutura controla o vetor dinâmico, a quantidade de registros válidos, a capacidade reservada e o próximo identificador. Na inicialização, a aplicação carrega primeiro o arquivo binário, caso exista; se não existir, importa o CSV. O menu principal encaminha o usuário para as operações de listagem, inclusão, atualização, remoção e busca. As buscas binárias são feitas por ID e nome; para cada chave, os dados são ordenados pela mesma chave antes da pesquisa. As alterações são gravadas durante as operações.

Ordem dos dados armazenados

Antes de gravar, a base em memória é ordenada crescentemente por ID. Os registros são escritos no clientes.dat nessa ordem, e a exportação CSV é produzida a partir da mesma base ordenada. Assim, os arquivos gerados normalmente ficam ordenados por ID. Ao carregar um clientes.dat existente, o programa o lê na ordem em que está e pressupõe que essa ordem já seja válida.

Acertos e pontos a corrigir

- Acertos: estrutura de registros adequada; vetor dinâmico implementado manualmente; Selection Sort em dois campos; buscas binárias recursivas; 100 cadastros no CSV; inclusão, alteração e remoção com gravação; identificadores e documentos validados; menus E/R coerentes com as ações.
- Pontos de atenção do enunciado disponível: o vetor inicia com capacidade 4 e dobra, embora a descrição peça espaço inicial para 40 e crescimento de 5 ou 10 posições; o arquivo DAT tem prioridade sobre a importação inicial do CSV; a gravação é automática, sem opção de confirmação; a listagem percorre páginas fixas de 10, sem receber início e fim do trecho pedido.
- Limites técnicos: o CSV implementa separação simples por ponto e vírgula, sem aspas escapadas; o arquivo binário depende da representação dos tipos e alinhamento da máquina; a limpeza da tela chama o comando externo clear em Linux. A matriz técnica, a seguir, associa cada ponto aos trechos concretos do código.

Conclusão

O sistema implementa as operações centrais de cadastro, ordenação, busca e persistência e demonstra alocação dinâmica manual e algoritmos próprios. A análise requisito por requisito também identifica diferenças entre o código atual e a descrição de projeto localizada. Essas diferenças devem ser avaliadas antes da entrega para que o grupo decida se adapta o programa ou justifica formalmente as escolhas.

Fonte dos requisitos: Projeto-Prático-Descrição-2025s2-1-4.pdf, localizado entre os materiais disponíveis. A matriz segue os sete requisitos numerados desse arquivo e acrescenta o mapeamento requisito-código solicitado pelo grupo. Como a descrição está identificada como 2025s2, compare os itens com qualquer versão mais recente publicada no Campus Virtual.

Este guia explica a versão revisada do sistema de gerenciamento de clientes. Ele foi preparado para você estudar o código e explicá-lo com suas próprias palavras. A meta não é decorar cada linha: é entender quais dados o programa guarda, como eles passam de uma função para outra e quando são gravados em arquivo.

Identificação do projeto: GAC124 - Introdução aos Algoritmos, turma 10R1. Discentes: Jhônata de Oliveira Marques e Henrique César Carvalho Aguiar, conforme o cabeçalho do main.cpp.

1. Visão geral do programa

O sistema funciona no terminal e oferece estas operações:

- listar clientes;
- cadastrar um cliente;
- atualizar ou remover um cadastro;
- buscar por ID ou por nome;
- carregar e salvar os dados em arquivos.

Durante a execução, os clientes ficam em um vetor dinâmico. O programa tenta carregar primeiro o arquivo binário clientes.dat. Se ele não existir, importa os dados de clientes.csv. Ao salvar, grava os registros no arquivo binário e também exporta um CSV.

Fluxo geral:

```
iniciar
  inicializar a base de clientes
  carregar os registros de arquivo
  repetir
    mostrar o menu
    ler a opção
    executar a operação correspondente
  até a opção de saída
  liberar a memória dinâmica
terminar
```

2. Relação com os conteúdos da disciplina

O plano de ensino fornecido prevê variáveis e constantes, condicionais, repetições, vetores, busca, ordenação, modularização, registros, arquivos, ponteiros e alocação dinâmica. O programa usa esses assuntos.

Isso indica relação com os conteúdos previstos, mas não prova que cada assunto ou sintaxe já tenha sido explicado em aula, nem que o professor espere exatamente esta implementação. O cronograma do plano coloca alguns conteúdos, como ordenação, redimensionamento e arquivos binários, em datas posteriores. Para falar da sequência de aprendizagem, considere o que foi efetivamente dado em aula e os requisitos do projeto.

3. Bibliotecas e começo do arquivo

As bibliotecas incluídas no início fornecem funcionalidades usadas ao longo do programa:

Biblioteca	Uso no código
iostream	cin, cout, cerr e endl para ler e escrever no terminal.
cstdio	perror, para informar erros de arquivos.
cstdlib	atoi, atof, system e NULL.
cstring	strcmp, comparação de textos em vetores de char.
fstream	ifstream e ofstream para leitura e gravação de arquivos.
string	string e operações como length, empty, find e c_str.

Biblioteca	Uso no código
iomanip	setw, left, fixed e setprecision para formatar a exibição.

using namespace std permite escrever string e cout em vez de std::string e std::cout. É uma conveniência de escrita; não é variável nem muda a lógica do programa.

Constantes

```
const char DATA_FILE[] = "clientes.dat";
const char CSV_FILE[] = "clientes.csv";
const int MAX_TEXT = 128;
```

const indica que o valor não deve ser alterado durante a execução.

- DATA_FILE é o nome do arquivo binário.
- CSV_FILE é o nome do arquivo CSV.
- MAX_TEXT é o tamanho reservado para os campos de texto nome e endereço.

Dar nomes a esses valores evita repetir literais pelo código e explica a finalidade de cada um. Se o limite dos campos mudar, por exemplo, pode-se alterar MAX_TEXT em um só lugar. O plano inclui constantes, então há respaldo no conteúdo programático para usá-las. Este é um ponto fácil de defender: diga o valor, o que ele representa e por que deve permanecer fixo.

4. Tipos de dados e registros

Registro Cliente

struct agrupa campos relacionados, mesmo quando eles têm tipos diferentes. Cada variável Cliente representa um cadastro.

Campo	Tipo	Por que esse tipo
id	int	Identificador inteiro do cliente.
nome_completo	char[128]	Texto em vetor de tamanho fixo.
endereco	char[128]	Texto em vetor de tamanho fixo.
ano_nascimento	short	Inteiro pequeno usado para o ano.
documento	char[32]	Texto com espaço limitado para CPF/CNPJ.
tipo_cliente	char	Uma letra, como F ou J.
sexo	char	Uma letra de opção.
estado_civil	char	Uma letra de opção.
limite_credito	float	Número que pode ter casas decimais.
situacao_cadastral	char	Uma letra, como A ou I.

Os campos textuais do registro são vetores de char, não string. O tamanho fixo torna o registro mais simples de gravar como um bloco binário. Esses textos precisam terminar com o caractere especial \0, que marca o fim da sequência. copiar_texto garante espaço para ele.

A ordem e os tipos dos campos fazem parte do formato de clientes.dat. Mudar os campos pode mudar o tamanho ou a interpretação dos registros existentes. A revisão manteve os campos do registro nessa mesma ordem.

Registro BaseClientes

BaseClientes guarda o estado do vetor dinâmico:

- dados: ponteiro para o primeiro Cliente do vetor;

- tamanho: quantidade de clientes efetivamente cadastrados;
- capacidade: quantidade de posições reservadas;
- proximo_id: ID usado no próximo cadastro;

Tamanho e capacidade são diferentes. Se tamanho vale 5 e capacidade vale 8, os índices 0 a 4 contêm registros válidos e há três posições reservadas ainda livres.

Em main, BaseClientes base = {}; inicia os campos numéricos em zero e o ponteiro como nulo. Depois base.proximo_id = 1 define o primeiro ID.

5. Funções, parâmetros e retornos

As declarações antecipadas, também chamadas de protótipos, anunciam funções que serão definidas mais abaixo. Isso permite chamar uma função antes de o compilador ter visto seu corpo completo.

O código usa:

- BaseClientes &base: referência à mesma base. Alterações feitas na função afetam o objeto original.
- const BaseClientes &base: referência de leitura, sem intenção de alterar a base e sem copiá-la.
- const string &texto: referência de leitura para uma string.
- Cliente *dados: ponteiro para elementos do vetor.
- const Cliente *dados: acesso de leitura aos registros apontados.

Retornos comuns:

- void: a função executa uma tarefa e não devolve valor;
- bool: devolve true ou false para indicar um resultado;
- int: devolve um índice ou -1 quando não há resultado;
- Cliente: devolve um registro preenchido.

6. Entrada e preparação de textos

arquivo_existe

Tenta abrir o caminho recebido com ifstream em modo binário e retorna teste.good(). O resultado informa se a abertura foi bem-sucedida. Isso é usado para decidir se o programa deve carregar clientes.dat.

ler_linha

Imprime o rótulo e lê uma linha completa com getline. É útil para nomes e endereços, que podem conter espaços.

ler_inteiro

Repete a leitura até obter um inteiro:

1. cin >> numero tenta converter a entrada para int.
2. cin.fail() verifica se a leitura falhou.
3. Se falhou, cin.clear() remove o estado de erro e cin.ignore(10000, '\n') descarta o resto da linha inválida.
4. Se deu certo, também descarta o restante da linha e retorna o número.

O descarte evita que o restante da entrada numérica atrapalhe a próxima leitura com getline.

ler_short e ler_float

ler_short reaproveita ler_inteiro e converte o valor para short. ler_float repete a validação de entrada usando uma variável float. O short tem faixa menor que int; o código não valida explicitamente se o ano cabe nessa faixa.

maiusculo e ler_char

maiusculo converte letras ASCII entre a e z para A e Z. Os comandos do menu podem assim aceitar s ou S, por exemplo. ler_char lê uma linha, retorna \0 se estiver vazia e, caso contrário, devolve a primeira letra em maiúscula. A função não valida se a opção pertence ao conjunto permitido; o menu faz essa verificação depois.

A conversão manual é adequada às letras ASCII das opções. Não é uma conversão completa de maiúsculas/minúsculas para todas as letras acentuadas em UTF-8.

copiar_texto

Recebe o destino, a origem e o limite disponível. O índice i conta quantos caracteres foram copiados. O laço para antes de ultrapassar limite - 1; no fim, escreve \0. Assim o campo permanece terminado e não ultrapassa seu vetor.

Um vetor char[128] comporta no máximo 127 caracteres de texto mais o terminador \0.

ler_campo

Lê um campo da linha CSV até encontrar ponto e vírgula ou chegar ao fim. posicao indica onde começar a leitura e é passada por referência para avançar entre chamadas. campo é esvaziado e preenchido com os caracteres do próximo campo.

7. Ponteiros, vetor dinâmico e memória

garantir_capacidade

Recebe a quantidade de posições necessárias. Se a capacidade existente já for suficiente, não muda nada e retorna true. Senão:

1. escolhe capacidade inicial 4 ou começa pela capacidade atual;
2. dobra capacidade_alvo até alcançar a quantidade pedida;
3. cria novo_buffer com new Cliente[capacidade_alvo];
4. copia nele os tamanho registros válidos;
5. libera o vetor antigo com delete[];
6. atualiza dados e capacidade.

novo_buffer guarda temporariamente o endereço da nova área. O índice i percorre apenas os clientes válidos. A capacidade pode ser maior do que tamanho para evitar reservar memória a cada inclusão.

Ponto importante desta versão: a função retorna bool, mas, quando termina normalmente, retorna sempre true. Ela não trata a falta de memória de new[] com uma recuperação própria. Assim, os if que verificam o retorno não são uma proteção contra falha de alocação.

destruir_base

Libera com delete[] a área que foi reservada com new[] e zera o ponteiro, tamanho e capacidade. É chamada no encerramento. O par correto é new[] com delete[].

8. Ordenação

ordenar_por_id

Implementa Selection Sort crescente por ID:

1. i marca a posição a preencher nesta rodada.
2. indice_min começa como a posição i.
3. j examina as posições seguintes em busca de um ID menor.
4. Ao encontrar um menor, atualiza indice_min.
5. Se o menor estiver em outra posição, temp guarda um dos registros enquanto os dois são trocados.

Depois de cada rodada, o menor ID do trecho não ordenado fica na posição i. O algoritmo é escrito com laços e comparações, sem uma função pronta de ordenação.

ordenar_por_nome

Também é Selection Sort, mas compara nome_completo com comparar_texto. Isso faz a ordenação seguir a mesma regra de comparação que a busca binária por nome.

comparar_texto

Percorre dois textos caractere a caractere, converte cada caractere ASCII para maiúscula e compara. Retorna -1 se o primeiro texto precede o segundo, 1 se sucede, e 0 se forem iguais segundo essa comparação. É usada pela busca binária por nome.

Ordenação e busca agora usam a mesma função de comparação, evitando que a busca binária descarte a região errada por causa de capitalização ASCII. A conversão manual só normaliza letras ASCII; não implementa regras linguísticas completas para letras acentuadas em UTF-8.

9. IDs e buscas

atualizar_proximo_id

Percorre os clientes e guarda em maior o maior ID encontrado. Define proximo_id como maior + 1. Se a base estiver vazia, maior continua 0 e o próximo ID fica 1.

Como calcula com base nos registros ainda existentes, se o maior ID for removido, esse número pode ser usado de novo. Não existe um histórico separado dos IDs já atribuídos.

encontrar_indice_por_id

Ordena a base por ID e chama busca_binaria_id. A ordenação é necessária porque busca binária depende de dados ordenados pela chave que está sendo procurada. Retorna o índice ou -1.

busca_binaria_id

busca_binaria_id inicia a busca no intervalo completo, da posição 0 até quantidade - 1, e chama busca_binaria_id_recursiva. Essa função recebe os limites do trecho que ainda pode conter o alvo. meio é o ponto central:

- se o ID central for o alvo, retorna meio;
- se o ID central for menor, chama a si mesma para a metade da direita;
- se for maior, chama a si mesma para a metade da esquerda;
- se inicio > fim, o intervalo acabou e a função retorna -1.

O caso inicio > fim é a condição de parada: sem ele, as chamadas poderiam continuar indefinidamente. Cada chamada escolhe apenas uma metade, então a quantidade de posições possíveis cai pela metade.

busca_binaria_nome

Ordena por nome o vetor recebido e inicia a busca recursiva no intervalo completo. busca_binaria_nome_recursiva compara o nome central com o termo usando comparar_texto; em seguida, retorna a posição, chama a si mesma em uma das metades ou retorna -1 quando o intervalo acaba. A ordenação pode alterar o vetor, por isso buscar_por_nome passa uma cópia e preserva a ordem da base original.

10. Arquivos

CSV e binário

clientes.csv é texto com colunas separadas por ponto e vírgula e pode ser inspecionado em um editor. clientes.dat armazena bytes de registros Cliente para leitura direta pelo programa.

O binário depende da representação dos tipos, do alinhamento dos campos e da arquitetura. É prático para o mesmo programa em ambiente compatível, mas não é automaticamente portátil entre compiladores e máquinas diferentes.

salvar_csv

Abre o arquivo com ios::trunc, que começa uma nova gravação apagando o conteúdo anterior. Escreve o cabeçalho e uma linha por cliente. fixed e setprecision(2) formatam o limite com duas casas decimais. A função verifica a abertura e o estado do arquivo após gravar.

importar_de_csv

Abre o CSV; se não conseguir, considera-o opcional e retorna true. Lê linha a linha. primeira permite reconhecer o cabeçalho id; na primeira linha. pular_linha decide se a linha será ignorada. Linhas vazias também são ignoradas.

Para cada linha de dados, posicao começa em zero. ler_campo retira os dez campos, um a um. atoi converte texto em inteiro; atof converte texto em número decimal. copiar_texto limita o tamanho dos campos textuais. O registro cli é então acrescentado ao vetor, ampliando-o se necessário.

O parser implementa um CSV simples: não trata campos entre aspas nem ponto e vírgula escapado dentro do texto. O formato esperado precisa seguir a estrutura simples gerada por salvar_csv.

carregar_clientes

A prioridade é:

1. Se clientes.dat existir, tenta abri-lo em modo binário.
2. Lê registros completos com read e acrescenta cada um ao vetor.
3. Atualiza o próximo ID e retorna.
4. Se não houver binário, importa do CSV.
5. Se a importação trouxe registros, chama salvar_clientes para criar o binário e exportar o CSV.

Se o binário existir, ele tem prioridade. O programa não verifica se o CSV está mais recente.

salvar_clientes

Ordena os registros por ID, abre clientes.dat em modo binário e truncado, e grava cada Cliente com write. A expressão (char *)&base.dados[i] fornece o endereço do registro como sequência de bytes. sizeof(Cliente) informa o número de bytes do registro.

Depois verifica erros e atualiza proximo_id. Em seguida chama salvar_csv. Se o binário foi salvo, mas a exportação do CSV falhar, mostra um aviso específico e considera a operação principal bem-sucedida: clientes.dat é o arquivo prioritário usado no próximo carregamento. As operações de inclusão, edição e remoção chamam essa função imediatamente.

11. Operações de cadastro

CRUD significa criar, consultar, atualizar e remover dados.

existe_documento

Percorre a base com i e compara o documento de cada registro com o documento recebido. Retorna true ao achar uma repetição; se terminar o laço, retorna false. É uma busca sequencial para evitar duplicidade.

ler_dados_cliente

Inicializa c zerado, define seu ID e pergunta os campos. nome, endereco e documento guardam temporariamente os textos lidos como string. copiar_texto os transfere com limites para os vetores char do registro. A função devolve o Cliente completo.

inserir_cliente

Usa proximo_id para montar o novo registro. Verifica se o documento já existe. Depois garante espaço, coloca novo na posição tamanho, aumenta tamanho e proximo_id e salva. Se a gravação retornar false, diminui tamanho e proximo_id para desfazer a inclusão em memória. Em sucesso, mostra a mensagem de conclusão.

atualizar_cliente

Lê o ID, procura seu índice e mostra erro se não existir. Lê os novos dados preservando o ID. O laço verifica se o documento novo já pertence a outro registro. mesmo_indice evita rejeitar o documento quando ele continua pertencendo ao próprio cliente que está sendo atualizado. Depois substitui o registro e salva.

remover_cliente

Lê o ID e localiza a posição. O laço copia cada registro seguinte uma posição para a esquerda. Decrementa tamanho e salva. A capacidade do vetor permanece igual, mas há um registro válido a menos.

editar_por_indice e remover_por_indice

São versões para quando a função chamadora já tem o índice do cliente. editar_por_indice repete a validação de documento e a substituição. remover_por_indice repete o deslocamento e a diminuição do tamanho. A repetição permite chamar essas ações pelo menu contextual além do menu principal.

manipular_cliente

Mostra ações para o registro indicado por indice. switch escolhe editar, remover, cadastrar outro ou voltar. O laço for (;;) repete quando a opção é inválida. Uma opção reconhecida retorna da função.

imprimir_cartao

Mostra os campos de um cliente. setw define larguras mínimas; left alinha o conteúdo à esquerda. fixed e setprecision(2) mostram o crédito com duas casas. Esses manipuladores afetam a apresentação, não os valores guardados.

listar_clientes

Mostra mensagem se não há clientes. Caso contrário, ordena por ID e pagina os resultados.

- por_pagina vale 10.
- indice marca o início da página.
- ate calcula o limite final e é reduzido se ultrapassar tamanho.
- i percorre a página.
- opcao guarda a resposta; acao guarda a primeira letra em maiúscula.
- P avança, E busca um ID e chama editar_por_indice, R busca um ID e chama remover_por_indice, N inicia cadastro e outras opções saem.

Assim, as ações descritas no prompt da listagem correspondem ao que acontece: E edita e R remove diretamente. O menu contextual continua disponível depois de uma busca individual.

buscar_por_id

Lê o ID, procura e, se encontrar, imprime o cartão e mostra o menu contextual. Se não encontrar, informa isso.

buscar_por_nome

Lê o termo, aloca uma cópia dinâmica da base e copia os clientes para ela. A busca ordena essa cópia por nome. Se encontrar um cadastro, usa seu ID para localizar o registro correspondente na base original antes de abrir o menu contextual. Ao terminar, libera a cópia com delete[].

12. Interface e função principal

pausar

Mostra uma instrução e ignora caracteres até a quebra de linha. Serve para dar tempo de ler a tela antes do próximo menu.

limpar_tela

O pré-processador verifica _WIN32: no Windows chama cls; caso contrário, clear. system executa o comando do sistema operacional. A função depende de esses comandos estarem disponíveis.

desenhar_banner e exibir_menu

desenhar_banner limpa a tela e imprime o título com separadores. exibir_menu chama essa função e escreve as opções. A modularização evita repetir o desenho do cabeçalho.

main

1. Cria a base zerada e inicia proximo_id em 1.
2. Chama carregar_clientes. Se houver falha, termina com retorno 1.
3. Inicia opcao em -1 para entrar no while.
4. Enquanto opcao não for 0, exibe o menu e lê a opção.
5. switch chama a função correspondente e pausa a tela nas opções de 1 a 6.
6. Na opção 0, o laço termina; cada operação de alteração já tentou gravar os arquivos.
7. Ao final, destrói a base e retorna 0.

13. Salvamento e saída

As operações de cadastro, edição e remoção chamam salvar_clientes imediatamente. A opção 0 agora encerra o laço sem apresentar uma segunda pergunta de salvamento. Isso deixa a saída coerente com o comportamento real: as alterações são persistidas durante as próprias operações. Se a gravação do binário funcionar e somente a atualização do CSV falhar, a mensagem informa que o registro está salvo no binário e que o CSV ficou desatualizado.

Se uma operação não conseguir salvar, ela informa a falha. A saída não tenta repetir automaticamente uma gravação que já falhou.

14. Pseudocódigo dos algoritmos centrais

Aumentar o vetor:

```
se a capacidade já for suficiente:
    terminar
escolher capacidade maior
criar novo vetor
copiar os registros válidos
liberar vetor antigo
atualizar ponteiro e capacidade
```

Selection Sort por ID:

```
para cada posição i até a penúltima:
    supor que i contém o menor ID restante
    procurar o menor ID entre i e o fim
    trocar o menor com a posição i
```

Busca binária:

```
busca_recursiva(inicio, fim):
    se inicio > fim: devolver -1
    meio = centro do intervalo
    se o elemento central é o alvo: devolver meio
    se o elemento central é menor: buscar recursivamente à direita
    caso contrário: buscar recursivamente à esquerda
```

Remover do vetor:

```
localizar o índice
deslocar os seguintes uma posição à esquerda
diminuir tamanho em 1
salvar os registros restantes
```

15. Perguntas prováveis

Por que há tamanho e capacidade? Tamanho conta os registros ocupados; capacidade conta as posições reservadas. Separá-los permite usar o espaço disponível sem realocar a cada inclusão.

Por que usar struct? Para agrupar os campos de tipos diferentes que descrevem o mesmo cliente.

Por que ordenar antes da busca binária? Porque a busca binária só descarta metades corretamente se o vetor estiver ordenado pela chave procurada.

Por que buscar o nome em uma cópia? Porque a busca por nome ordena o vetor. A cópia preserva a ordem da base principal para outras operações.

Por que as constantes no começo? Elas dão nomes a valores fixos, centralizam configurações e evitam números e nomes de arquivo repetidos. A disciplina inclui constantes.

Por que getline para o nome? Para receber a linha inteira, incluindo espaços.

Para que servem cin.clear e cin.ignore? clear permite novas leituras após uma falha; ignore descarta a entrada inválida que restou na linha.

O que significa -1 na busca? É um indicador de que nenhum índice válido foi encontrado.

Como a busca recursiva para? Quando inicio fica maior que fim, não há posições restantes para verificar. Essa é a condição de parada, e o resultado é -1.

Por que new[] e delete[]? O vetor é reservado dinamicamente porque sua capacidade pode crescer. delete[] libera a área no encerramento ou quando ela é substituída.

CSV e binário são a mesma coisa? Não. CSV é texto delimitado, binário é uma sequência de bytes na representação do registro.

O binário é portátil? Não necessariamente. Depende dos tamanhos e da representação dos tipos e do alinhamento usados pelo compilador e pela máquina.

Como a ordenação por nome combina com a busca binária? As duas usam comparar_texto, então aplicam a mesma regra a letras ASCII e a busca pode confiar na ordem produzida. A comparação manual não aplica conversão linguística completa a letras acentuadas em UTF-8.

Quando as alterações são salvas? As operações CRUD tentam salvar imediatamente. Ao escolher 0, o programa encerra sem uma pergunta redundante de salvamento.

16. Roteiro para apresentar

Uma apresentação pode seguir estes passos:

1. Objetivo: explicar que o programa gerencia cadastros no terminal.
2. Representação: mostrar Cliente como registro e BaseClientes como controle do vetor.
3. Entrada: explicar getline, validação numérica e cópia limitada dos textos.
4. Memória: descrever tamanho, capacidade, new[], cópia e delete[].
5. Algoritmos: explicar Selection Sort e busca binária.
6. Persistência: comparar CSV texto com DAT binário e mostrar a prioridade de carregamento.
7. Operações: resumir inclusão, busca, atualização e remoção.
8. Limitações: reconhecer o parser CSV simples, a dependência do formato binário da máquina e o tratamento limitado de falta de memória.

É melhor explicar a lógica com um exemplo pequeno do que ler o código linha a linha. Por exemplo, para a busca binária, use cinco IDs ordenados e mostre como o intervalo diminui.

16.1 Como justificar uma escolha diferente

Você não precisa concordar automaticamente com toda preferência de estilo. Na apresentação, a justificativa fica mais forte quando explica o objetivo, o benefício, o custo e a alternativa. Uma forma respeitosa de responder é:

1. reconhecer a preocupação do professor;

2. explicar o que você queria melhorar para o usuário;
3. esclarecer se a escolha afeta a lógica principal ou apenas a interface;
4. reconhecer a limitação da sua escolha;
5. dizer em que condição você aceitaria outra solução.

Exemplo: limpar a tela e redesenhar o banner

O código tem a função limpar_tela, que chama cls no Windows ou clear nos outros sistemas, e desenhar_banner, que limpa a tela e imprime um título entre separadores. exibir_menu usa esse banner antes de mostrar as opções.

Uma justificativa possível para manter esse recurso:

> “Eu entendo a preferência por uma interface de terminal mais simples. Mantive a limpeza e o banner porque organizam a tela, destacam o menu atual e evitam que várias telas de opções fiquem acumuladas. Isso ajuda o usuário a acompanhar o fluxo do sistema. A escolha afeta apenas a apresentação; não altera os registros, as buscas ou a persistência dos dados.”

Também é importante reconhecer o custo:

> “A limpeza usa comandos específicos do sistema operacional e apaga o conteúdo anterior do terminal. Se a prioridade for máxima portabilidade, simplicidade ou manter o histórico visível, eu removeria a chamada a limpar_tela ou deixaria o banner sem limpar a tela.”

Essa defesa não afirma que a escolha é a única correta. Ela mostra que houve uma decisão consciente entre organização visual e simplicidade/portabilidade. Não diga que o uso de system ou dos comandos clear/cls foi ensinado na disciplina sem ter certeza; o plano de ensino lista os conteúdos de algoritmos, mas não estabelece essa escolha de interface.

Para outras críticas, use o mesmo formato. Não invente uma justificativa antes de saber exatamente qual é a observação e qual requisito ela atende.

17. Compilar e executar

Compile:

```
g++ -std=c++11 -Wall -Wextra -pedantic main.cpp -o main
```

Execute dentro do diretório que contém os arquivos de dados:

```
./main
```

O programa procura clientes.dat e clientes.csv no diretório de execução atual, não necessariamente na pasta onde main.cpp está salvo.

18. Glossário

Termo	Significado
Registro	Agrupamento de campos diferentes, como Cliente.
Vetor	Sequência de elementos acessados por índices começando em zero.
Vetor dinâmico	Vetor alocado durante a execução, que pode ser trocado por outro maior.
Ponteiro	Variável que guarda um endereço; dados aponta para o vetor.
Índice	Posição de um elemento no vetor.
Capacidade	Número de posições reservadas.
Selection Sort	Ordenação que escolhe o menor elemento restante e o troca para a posição atual.
Busca binária	Busca que reduz o intervalo possível pela metade a cada comparação.
Persistência	Gravação de dados para que sobrevivam ao encerramento.
CSV	Arquivo de texto com campos separados por delimitadores.
Binário	Arquivo que guarda bytes do registro na representação do programa.
CRUD	Criar, consultar, atualizar e remover.
Referência	Parâmetro que dá acesso ao objeto original.
\0	Caractere que marca o fim de um texto em vetor de char.

Matriz de rastreabilidade: requisitos técnicos e código

Cada item abaixo resume um requisito do enunciado de projeto disponível, indica o atendimento encontrado e mostra o recorte literal do main.cpp que permite conferir a análise. Os números à esquerda são as linhas atuais do arquivo incluído no pacote. Os status distinguem atendimento direto, atendimento parcial e divergência.

LEGENDA	ATENDE: o trecho demonstra o requisito. PARCIAL: há parte da funcionalidade, mas o comportamento difere em algum ponto. DIVERGÊNCIA: o comportamento atual não corresponde ao requisito resumido.

1. Carregamento inicial e dimensionamento do vetor

PARCIAL	Há vetor dinâmico e importação CSV de reserva, mas a capacidade e a prioridade de carregamento diferem do enunciado.

O que o enunciado pede: Carregar inicialmente o CSV em vetor dinâmico, começar com capacidade para 40 elementos e aumentar a capacidade em blocos de 5 ou 10 posições.

Trecho de main.cpp, linhas 358-385: carregar_clientes: DAT tem prioridade; CSV é fallback.

```
0358 | bool carregar_clientes(BaseClientes &base) {
0359 |     if (arquivo_existe(DATA_FILE)) {
0360 |         ifstream in(DATA_FILE, ios::binary);
0361 |         if (!in) {
0362 |             perror("Não foi possível abrir o arquivo de dados");
0363 |             return false;
0364 |         }
0365 |
0366 |         Cliente temp = {};
0367 |         while (in.read((char *)&temp, sizeof(Cliente))) {
0368 |             if (!garantir_capacidade(base, base.tamanho + 1)) {
0369 |                 return false;
0370 |             }
0371 |             base.dados[base.tamanho++] = temp;
0372 |         }
0373 |         atualizar_proximo_id(base);
0374 |         return true;
0375 |     }
0376 |
0377 |     if (!importar_de_csv(base)) {
0378 |         return false;
0379 |     }
0380 |
0381 |     if (base.tamanho == 0) {
0382 |         return true;
0383 |     }
0384 |
0385 |     return salvar_clientes(base);
```

Trecho de main.cpp, linhas 166-185: garantir_capacidade: capacidade inicial 4 e duplicação.

```
0166 | bool garantir_capacidade(BaseClientes &base, int nova_capacidade) {
0167 |     if (nova_capacidade <= base.capacidade) {
0168 |         return true;
0169 |     }
0170 |
0171 |     int capacidade_alvo = base.capacidade == 0 ? 4 : base.capacidade;
0172 |     while (capacidade_alvo < nova_capacidade) {
0173 |         capacidade_alvo *= 2;
0174 |     }
0175 |
0176 |     Cliente *novo_buffer = new Cliente[capacidade_alvo];
0177 |
0178 |     for (int i = 0; i < base.tamanho; ++i) {
0179 |         novo_buffer[i] = base.dados[i];
0180 |     }
0181 |
0182 |     delete[] base.dados;
0183 |     base.dados = novo_buffer;
0184 |     base.capacidade = capacidade_alvo;
0185 |     return true;
```

Leitura do código: carregar_clientes procura clientes.dat primeiro e só importa clientes.csv se o binário não existir. garantir_capacidade começa em 4 e dobra sucessivamente (4, 8, 16, 32, 64...). Isso demonstra alocação dinâmica, mas não atende ao início em 40 nem aos incrementos de 5 ou 10; no pacote atual, como

clientes.dat está presente, o CSV inicial não é a fonte lida na primeira execução.

2. Inserção de um novo elemento

ATENDE	O registro novo é colocado no fim do trecho ocupado do vetor.

O que o enunciado pede: Permitir inserir um elemento no final do vetor; inserção ordenada é opcional.

Trecho de main.cpp, linhas 561-586: inserir_cliente: validação, posição final, atualização do tamanho e gravação.

```
0561 | bool inserir_cliente(BaseClientes &base) {
0562 |     desenhar_banner("Novo cadastro de cliente");
0563 |     Cliente novo = ler_dados_cliente(base.proximo_id);
0564 |     if (existe_documento(base, novo.documento)) {
0565 |         cout << endl << "Documento " << novo.documento << " já cadastrado." << endl << endl;
0566 |         return false;
0567 |     }
0568 |
0569 |     if (!garantir_capacidade(base, base.tamanho + 1)) {
0570 |         cerr << endl << "Não há memória suficiente para novos cadastros." << endl << endl;
0571 |         return false;
0572 |     }
0573 |
0574 |     base.dados[base.tamanho++] = novo;
0575 |     base.proximo_id++;
0576 |     if (!salvar_clientes(base)) {
0577 |         base.tamanho--;
0578 |         base.proximo_id--;
0579 |         cerr << endl
0580 |              << "Cadastro desfeito: não foi possível salvar por falta de espaço ou erro de gravação." << endl
0581 |              << endl;
0582 |         return false;
0583 |     }
0584 |
0585 |     cout << endl << "Cliente cadastrado com sucesso!" << endl << endl;
0586 |     return true;
```

Leitura do código: O tamanho atual é usado como índice de destino. Depois da atribuição, tamanho aumenta. A operação então chama salvar_clientes; a ordenação por ID acontece antes da gravação, sem alterar o fato de que a inclusão em memória foi feita no final.

3. Remoção de dados existentes

ATENDE	A remoção desloca os registros seguintes e a gravação reescreve o arquivo sem o item removido.

O que o enunciado pede: Permitir remover um cadastro e garantir que ele não permaneça no arquivo após uma gravação.

Trecho de main.cpp, linhas 618-637: remover_cliente: deslocamento, redução do tamanho e persistência.

```
0618 | bool remover_cliente(BaseClientes &base) {
0619 |     desenhar_banner("Remover cliente");
0620 |     int id = ler_inteiro("Informe o ID para exclusão");
0621 |     int indice = encontrar_indice_por_id(base, id);
0622 |     if (indice < 0) {
0623 |         cout << endl << "Cliente não encontrado." << endl << endl;
0624 |         return false;
0625 |     }
0626 |
0627 |     for (int i = indice; i + 1 < base.tamanho; ++i) {
0628 |         base.dados[i] = base.dados[i + 1];
0629 |     }
0630 |     --base.tamanho;
0631 |
0632 |     if (!salvar_clientes(base)) {
0633 |         return false;
0634 |     }
0635 |
0636 |     cout << endl << "Registro removido." << endl << endl;
0637 |     return true;
```

Leitura do código: A função localiza o índice, copia cada registro posterior uma posição para a esquerda, diminui tamanho e chama salvar_clientes. Como salvar_clientes abre o DAT com truncamento e grava apenas os registros válidos, o removido deixa de constar no arquivo.

4. Opção para gravar alterações antes do encerramento

PARCIAL	Os dados são salvos automaticamente após as operações, mas não há escolha explícita de salvar ou descartar.

O que o enunciado pede: Antes do término, permitir gravar inclusões e remoções; o enunciado sugere menu ou pergunta após as operações.

Trecho de main.cpp, linhas 574-585: inserção chama salvar_clientes.

```
0574 |     base.dados[base.tamanho++] = novo;
0575 |     base.proximo_id++;
0576 |     if (!salvar_clientes(base)) {
0577 |         base.tamanho--;
0578 |         base.proximo_id--;
0579 |         cerr << endl
0580 |              << "Cadastro desfeito: não foi possível salvar por falta de espaço ou erro de gravação." << endl
0581 |              << endl;
0582 |         return false;
0583 |     }
0584 |
0585 |     cout << endl << "Cliente cadastrado com sucesso!" << endl << endl;
```

Trecho de main.cpp, linhas 609-615: atualização chama salvar_clientes.

```
0609 |     base.dados[indice] = atualizado;
0610 |     if (!salvar_clientes(base)) {
0611 |         return false;
0612 |     }
0613 |
0614 |     cout << endl << "Registro atualizado com sucesso!" << endl << endl;
0615 |     return true;
```

Trecho de main.cpp, linhas 627-637: remoção chama salvar_clientes.

```
0627 |     for (int i = indice; i + 1 < base.tamanho; ++i) {
0628 |         base.dados[i] = base.dados[i + 1];
0629 |     }
0630 |     --base.tamanho;
0631 |
0632 |     if (!salvar_clientes(base)) {
0633 |         return false;
0634 |     }
0635 |
0636 |     cout << endl << "Registro removido." << endl << endl;
0637 |     return true;
```

Trecho de main.cpp, linhas 809-815: opção 0 encerra o switch sem confirmação de salvamento.

```
0809 |             case 0:
0810 |                 break;
0811 |             default:
0812 |                 cout << "Opção inválida!" << endl << endl;
0813 |                 pausar();
0814 |                 break;
0815 |         }
```

Leitura do código: O sistema grava imediatamente após inclusão, atualização e remoção. Isso reduz a chance de perder alterações, mas a opção 0 apenas encerra o laço e libera a memória: o usuário não escolhe se grava ou descarta. Portanto a persistência existe, mas a escolha solicitada não está disponível.

5. Busca de registros por ao menos dois campos com busca binária

ATENDE	Há busca binária recursiva por ID e por nome, com ordenação pela chave pesquisada.

O que o enunciado pede: Permitir buscar por campos diferentes; usar busca binária em cada campo após ordenar os dados pela chave correspondente.

Trecho de main.cpp, linhas 260-263: localização por ID ordena antes de pesquisar.

```
0260 | int encontrar_indice_por_id(BaseClientes &base, int id) {
0261 |     ordenar_por_id(base.dados, base.tamanho);
0262 |     return busca_binaria_id(base.dados, base.tamanho, id);
0263 | }
```

Trecho de main.cpp, linhas 422-438: busca binária recursiva por ID.

```
0422 | int busca_binaria_id(const Cliente *dados, int quantidade, int alvo) {
0423 |     return busca_binaria_id_recursiva(dados, 0, quantidade - 1, alvo);
0424 | }
0425 |
0426 | int busca_binaria_id_recursiva(const Cliente *dados, int inicio, int fim, int alvo) {
0427 |     if (inicio > fim) {
0428 |         return -1;
0429 |     }
0430 |
0431 |     int meio = inicio + (fim - inicio) / 2;
0432 |     if (dados[meio].id == alvo) {
0433 |         return meio;
0434 |     }
0435 |     if (dados[meio].id < alvo) {
0436 |         return busca_binaria_id_recursiva(dados, meio + 1, fim, alvo);
0437 |     }
0438 |     return busca_binaria_id_recursiva(dados, inicio, meio - 1, alvo);
```

Trecho de main.cpp, linhas 441-459: ordenação e busca recursiva por nome.

```
0441 | int busca_binaria_nome(Cliente *dados, int quantidade, const string &nome) {
0442 |     ordenar_por_nome(dados, quantidade);
0443 |     return busca_binaria_nome_recursiva(dados, 0, quantidade - 1, nome);
0444 | }
0445 |
0446 | int busca_binaria_nome_recursiva(const Cliente *dados, int inicio, int fim, const string &nome) {
0447 |     if (inicio > fim) {
0448 |         return -1;
0449 |     }
0450 |
0451 |     int meio = inicio + (fim - inicio) / 2;
0452 |     int comparacao = comparar_texto(dados[meio].nome_completo, nome.c_str());
0453 |     if (comparacao == 0) {
0454 |         return meio;
0455 |     }
0456 |     if (comparacao < 0) {
0457 |         return busca_binaria_nome_recursiva(dados, meio + 1, fim, nome);
0458 |     }
0459 |     return busca_binaria_nome_recursiva(dados, inicio, meio - 1, nome);
```

Leitura do código: encontrar_indice_por_id ordena por ID antes da busca. busca_binaria_id_recursiva usa o intervalo e reduz pela metade. A busca por nome ordena a cópia recebida por nome e chama seu auxiliar recursivo; comparar_texto é a mesma regra usada pela ordenação e pela pesquisa.

6. Exibição do arquivo inteiro ou de um trecho escolhido pelo usuário

PARCIAL	A listagem percorre todos os registros em páginas de 10, mas não solicita ao usuário os limites de um intervalo arbitrário.

O que o enunciado pede: Mostrar todos os registros ou um trecho indicado pelo usuário, preservando a ordem armazenada.

Trecho de main.cpp, linhas 513-535: listar_clientes: ordem por ID, páginas fixas e avanço sequencial.

```
0513 |     ordenar_por_id(base.dados, base.tamanho);
0514 |     const int por_pagina = 10;
0515 |     int indice = 0;
0516 |
0517 |     while (indice < base.tamanho) {
0518 |         int ate = indice + por_pagina;
0519 |         if (ate > base.tamanho) {
0520 |             ate = base.tamanho;
0521 |         }
0522 |
0523 |         cout << "Mostrando registros " << (indice + 1) << " a " << ate << " de " << base.tamanho << endl << endl;
0524 |         for (int i = indice; i < ate; ++i) {
0525 |             imprimir_cartao(base.dados[i]);
0526 |         }
0527 |
0528 |         string opcao = ler_linha("[P]róxima página, [E]ditar ID, [R]emover ID, [N]ovo cadastro, [S]air: ");
0529 |         if (!opcao.empty()) {
0530 |             char acao = maiusculo(opcao[0]);
0531 |             if (acao == 'P') {
0532 |                 indice = ate;
0533 |                 if (indice >= base.tamanho) {
0534 |                     return;
0535 |                 }
```

Leitura do código: A função ordena por ID e mostra páginas numeradas de 10 em 10. O usuário pode avançar para percorrer a base inteira, mas não informa, por exemplo, que deseja começar no registro 10 e

terminar no 20. A listagem também ordena a base por ID antes de exibi-la; os arquivos gerados pelo salvamento já são gravados nessa ordem.

7. Ordenação por pelo menos dois campos em memória

ATENDE	Selection Sort manual organiza os registros por ID e por nome; não há bubblesort nem std::sort.

O que o enunciado pede: Ordenar em memória por dois campos, utilizando método apresentado em aula e evitando bubblesort e variantes.

Trecho de main.cpp, linhas 192-206: Selection Sort por ID.

```
0192 | void ordenar_por_id(Cliente *dados, int quantidade) {
0193 |     for (int i = 0; i + 1 < quantidade; ++i) {
0194 |         int indice_min = i;
0195 |         for (int j = i + 1; j < quantidade; ++j) {
0196 |             if (dados[j].id < dados[indice_min].id) {
0197 |                 indice_min = j;
0198 |             }
0199 |         }
0200 |         if (indice_min != i) {
0201 |             Cliente temp = dados[i];
0202 |             dados[i] = dados[indice_min];
0203 |             dados[indice_min] = temp;
0204 |         }
0205 |     }
0206 | }
```

Trecho de main.cpp, linhas 230-244: Selection Sort por nome.

```
0230 | void ordenar_por_nome(Cliente *dados, int quantidade) {
0231 |     for (int i = 0; i + 1 < quantidade; ++i) {
0232 |         int indice_min = i;
0233 |         for (int j = i + 1; j < quantidade; ++j) {
0234 |             if (comparar_texto(dados[j].nome_completo, dados[indice_min].nome_completo) < 0) {
0235 |                 indice_min = j;
0236 |             }
0237 |         }
0238 |         if (indice_min != i) {
0239 |             Cliente temp = dados[i];
0240 |             dados[i] = dados[indice_min];
0241 |             dados[indice_min] = temp;
0242 |         }
0243 |     }
0244 | }
```

Leitura do código: ordenar_por_id e ordenar_por_nome são Selection Sort implementados com laços, escolha do menor elemento restante e troca. A busca por nome usa a mesma comparação textual que a ordenação. O plano de ensino disponível cita ordenação interna, mas não identifica qual método foi mostrado em cada aula; a confirmação do método em sala depende do material da turma.

Requisitos gerais e evidências complementares

Estrutura de dados, tipos e quantidade de registros

O enunciado pede pelo menos cinco informações, pelo menos dois tipos, um campo textual, um numérico e um texto que aceite espaços. Cliente contém dez campos, combinando int, short, float, char e vetores de char. A conferência do CSV incluído encontrou 100 registros de dados e 10 colunas; nome e endereço contêm espaços.

Trecho de main.cpp, linhas 25-36: estrutura Cliente com dez campos.

```
0025 | struct Cliente {
0026 |     int id;
0027 |     char nome_completo[MAX_TEXT];
0028 |     char endereco[MAX_TEXT];
0029 |     short ano_nascimento;
0030 |     char documento[32];
0031 |     char tipo_cliente;
0032 |     char sexo;
0033 |     char estado_civil;
0034 |     float limite_credito;
0035 |     char situacao_cadastral;
0036 | };
```

Exemplo do arquivo CSV incluído (linhas 1-2): 100 registros de dados, 10 campos por registro.

```
0001 | id;nome_completo;endereco;ano_nascimento;documento;tipo_cliente;sexo;estado_civil;limite_credito;situacao_cadastral
0002 | 1;Mário Rafael Souza;Rua Roraima 291, Fortaleza;1959;10000000001;F;O;S;33996.62;A
```

C++, Linux e restrições de implementação

A aplicação está em C++ e foi compilada com g++ em modo C++11, sem avisos. A inspeção não encontrou goto, continue, std::vector, <vector> ou std::sort. As ocorrências de break estão dentro de switch. O uso manual de new[]/delete[] e os algoritmos próprios demonstram os tópicos de alocação dinâmica, ordenação e busca.

Trecho de main.cpp, linhas 159-185: liberação e redimensionamento manual do vetor.

```
0159 | void destruir_base(BaseClientes &base) {
0160 |     delete[] base.dados;
0161 |     base.dados = NULL;
0162 |     base.tamanho = 0;
0163 |     base.capacidade = 0;
0164 | }
0165 |
0166 | bool garantir_capacidade(BaseClientes &base, int nova_capacidade) {
0167 |     if (nova_capacidade <= base.capacidade) {
0168 |         return true;
0169 |     }
0170 |
0171 |     int capacidade_alvo = base.capacidade == 0 ? 4 : base.capacidade;
0172 |     while (capacidade_alvo < nova_capacidade) {
0173 |         capacidade_alvo *= 2;
0174 |     }
0175 |
0176 |     Cliente *novo_buffer = new Cliente[capacidade_alvo];
0177 |
0178 |     for (int i = 0; i < base.tamanho; ++i) {
0179 |         novo_buffer[i] = base.dados[i];
0180 |     }
0181 |
0182 |     delete[] base.dados;
0183 |     base.dados = novo_buffer;
0184 |     base.capacidade = capacidade_alvo;
0185 |     return true;
```

Trecho de main.cpp, linhas 784-815: os break aparecem dentro do switch do menu principal.

```
0784 |         switch (opcao) {
0785 |             case 1:
0786 |                 listar_clientes(base);
0787 |                 pausar();
0788 |                 break;
0789 |             case 2:
0790 |                 inserir_cliente(base);
0791 |                 pausar();
0792 |                 break;
0793 |             case 3:
0794 |                 atualizar_cliente(base);
0795 |                 pausar();
0796 |                 break;
0797 |             case 4:
0798 |                 remover_cliente(base);
0799 |                 pausar();
0800 |                 break;
0801 |             case 5:
0802 |                 buscar_por_id(base);
0803 |                 pausar();
0804 |                 break;
0805 |             case 6:
0806 |                 buscar_por_nome(base);
0807 |                 pausar();
0808 |                 break;
0809 |             case 0:
0810 |                 break;
0811 |             default:
0812 |                 cout << "Opção inválida!" << endl << endl;
0813 |                 pausar();
0814 |                 break;
0815 |         }
```

Trecho de main.cpp, linhas 746-752: limpeza de tela: depende do comando externo clear/cls.

```
0746 | void limpar_tela() {
0747 |     #ifdef _WIN32
0748 |     system("cls");
0749 |     #else
0750 |     system("clear");
0751 |     #endif
0752 | }
```

A compilação em Linux foi verificada; ainda assim, system("clear") depende de o comando clear existir e de haver terminal compatível. A portabilidade do armazenamento binário também depende dos tipos, alinhamento e arquitetura. São limitações a descrever no relatório, não evidência de falha na compilação feita.

Cabeçalho, grupo e forma da entrega

O cabeçalho do fonte identifica disciplina, turma, dois discentes e tema. O pacote contém o código-fonte, o CSV inicial, o guia e o relatório em PDF. A descrição disponível também recomenda que o nome do arquivo compactado inclua os primeiros nomes dos integrantes e o trabalho; o nome do pacote final deve seguir essa convenção.

Trecho de main.cpp, linhas 11-19: identificação e tema no cabeçalho do fonte.

```
0011 | // ============================================================
0012 | // Sistema de Gerenciamento de Clientes
0013 | // Disciplina: GAC124 - Introdução aos Algoritmos
0014 | // Turma: 10R1
0015 | // Discentes: Jhônata de Oliveira Marques e Henrique César Carvalho Aguiar
0016 | // Data: 2026/2
0017 | // Descrição: Aplicação de terminal para gerenciamento de clientes
0018 | // com persistência em arquivo binário ordenado.
0019 | // ============================================================
```

Como o relatório atende ao formato solicitado

A descrição de projeto disponível pede uma introdução breve, explicação de alto nível das estruturas sem código, ordem dos dados, acertos e erros e conclusão. Essas partes aparecem na seção Relatório formal do trabalho. A matriz deste anexo foi separada da narrativa para incluir os trechos de código requisito por requisito conforme a orientação de apresentação do Campus Virtual comunicada pelo grupo.

Referências e limites da análise

- Código-fonte revisado: main.cpp, identificado no cabeçalho como projeto GAC124, turma 10R1.
- Descrição do projeto prático: Projeto-Prático-Descrição-2025s2-1-4.pdf; a matriz segue os sete requisitos técnicos numerados desse arquivo.
- Guia de estudo: guia_de_estudo_main_cpp.md, fonte da explicação função por função incluída neste relatório.
- Plano de Ensino GAC124 — Introdução aos Algoritmos: conteúdos de busca binária e chamadas recursivas, além de vetores, registros, arquivos, modularização e alocação dinâmica.
- As afirmações sobre funcionamento descrevem a versão revisada do código. Os testes citados cobrem compilação e funções de busca; outras operações são explicadas por leitura do código e não foram todas exercitadas de ponta a ponta nesta etapa.

O relatório é material de estudo e apresentação do projeto. Ele explica as decisões identificáveis no código e também registra limites técnicos que podem surgir em perguntas, como o parser CSV simples, a dependência do formato binário e a limpeza de tela por comandos do sistema operacional.